Manil Bastola
Stephanie Mertz
CSE 566S
Homework 5

All experiments in this paper were performed on Wustl Linux Lab server.

1.) What is the performance of the CPU code vs. the GPU code? Try to use similar parameters for each program when you compare, and explain why the performance differs, if necessary.

We hypothesized that for simple arithmetic based program as nbody simulation, the GPU code using cuda would be much faster than any CPU based parallelization. This is because the program can be massively parallelized since each body pair lacks the need to communicate and share intermediate outputs with other bodies for force calculation. Hence the need for need for shared memory/resources is minimal.

We took 6 sets of observations setting N = 30000, dt = 0.01 and numIter = 10 for both programs. For the CPU version, the OpenMP numthreads was set to 16 since this was the thread count that performed the best with the N-body problem in the previous lab. The runtime we observed for this program was 12.665 ± 0.015 seconds which pertains to 0.640 ± 0.001 Billion Interactions/second.

Using GPU, we set the block size to 512 with nBlocks being calculated using the formula (nBodies + BLOCK_SIZE - 1) / BLOCK_SIZE . The performance of GPU code was 0.305 ± 0.06s which corresponds to 27.085 ± 0.213 Billion Interactions/second. The performance gain observed was ~ 42X.

The performance of GPU is higher as expected. Since GPU can utilize higher number of cores, it was expected to perform better. For example, the lab server GTX750 has 512 cores. Our computation was all done on the GPU and CPU cycles were not utilized. So although CPU has 16 cores we only used 1 for the GPU. Hence we would expect 512/16 = 32X speedup. The program performed better. Because of low communication overhead, the GPU was able to optimize the execution more than the CPU.

2.) Investigate how changing the block size and thread count parameters affect code performance. Is there a relationship between the two?

We set N=30000 and varied BlockSizes vs numBlocks in the range 1 to 16k to observe the relationship between the two. The experiment was repeated 6 times. We plotted a 3D map to show the performace dependencies on both BlockSize and thread count. The original data can be found in the Appendix.

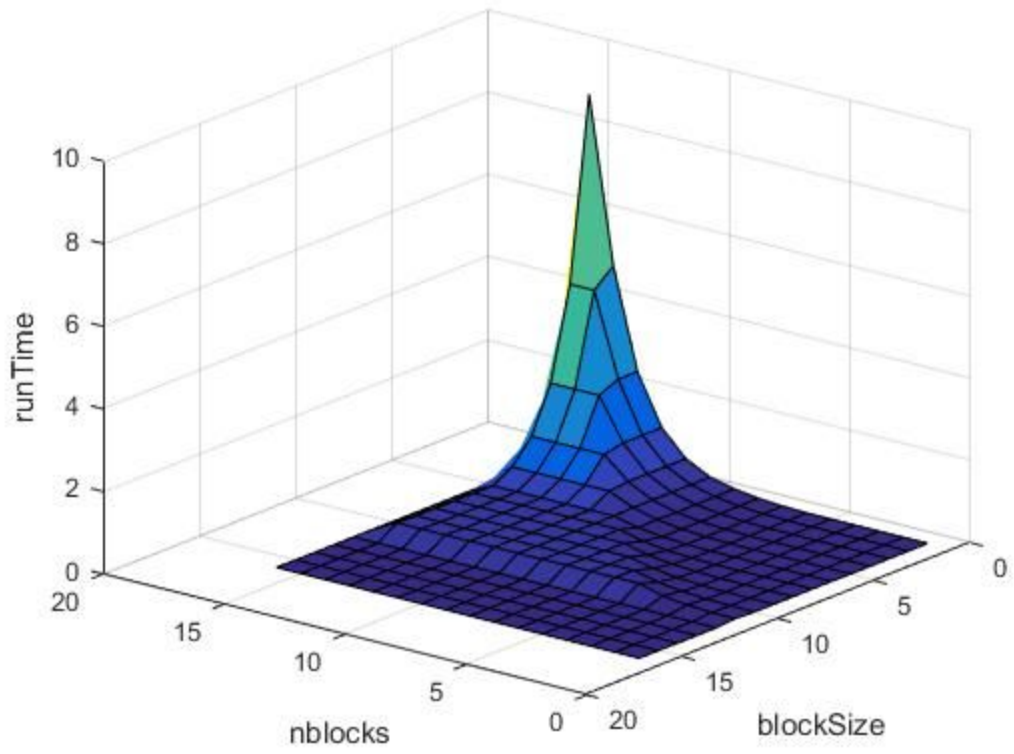


Figure 1: Cuda Program Runtime vs BlockSize and nBlocks (2^axis values).

The plot is very interesting and also misleading. We can observe constant performace and a sharp peak at BlockSize > 2^10 and numBlocks < 2^6. Aside from the peak, there is a

The data with block size <2^5 is not a sound observation since a warp is minimum of 32 threads. Hence, anything below 32 was probably still running 32 threads even if if we specified 16 and might have had the remaining 16 threads doing nothing. In our observation we see that such data-points had really low running time indicating early stopping/loss of threads.

Additionally, block size > 2^10 are also not sound data because on GTX the compute capability (2.x or 3.x) has a max threads per block of 1024. When we run it with anything greater, it likely confuses the GPU causing it to immediately end. This explains why we see such a tiny performance time when we make block size larger than 2^11.

Finally, the datapoints where the combination of block size and number of blocks is less than number of bodies ie. 30000 can also be biased. This is because the gpu assigns each body and id once before it puts them to the job pipeline. So once a thread is completed, the program may not pick up the bodies which were left out since they did not have ids in the first instance. Because of this, numBlocks times block size <= N may may have caused early termination

without taking all the bodies into account and the time for the code to complete was unfairly shortened.
This narrows down the data in the range where block size * number of blocks is as close to number of bodies when we observe on the plateau i.e. 2^5<= nblocks <2^12 and 2^4< blockSize < 2^11. We think the flat plateau region (average run time value of 0.302 ± 0.024 s) can be explained as follows: Because there is no communication cost and the overhead of GPUs is very minimal in the range, we did not see performance degrade too much since extra threads were “effectively” taken care in the pipeline. This means that the GPU was able to complete its current warps before the new threads had any time to stall..

3.) How does the size of the problem affect performance? Does it affect performance the same way on the GPU as it does on the CPU? Why or why not?
For this problem we varied N keeping the blockSize constant to 256. The numblocks parameter changed as (N + BLOCK_SIZE - 1) / BLOCK_SIZE. We hypothesized that the GPU would be able to keep the growth fairly linear till a certain extent beyond which quadratic effects would dominate.Hence, the asymptotic running time would reach back to O(n^2).

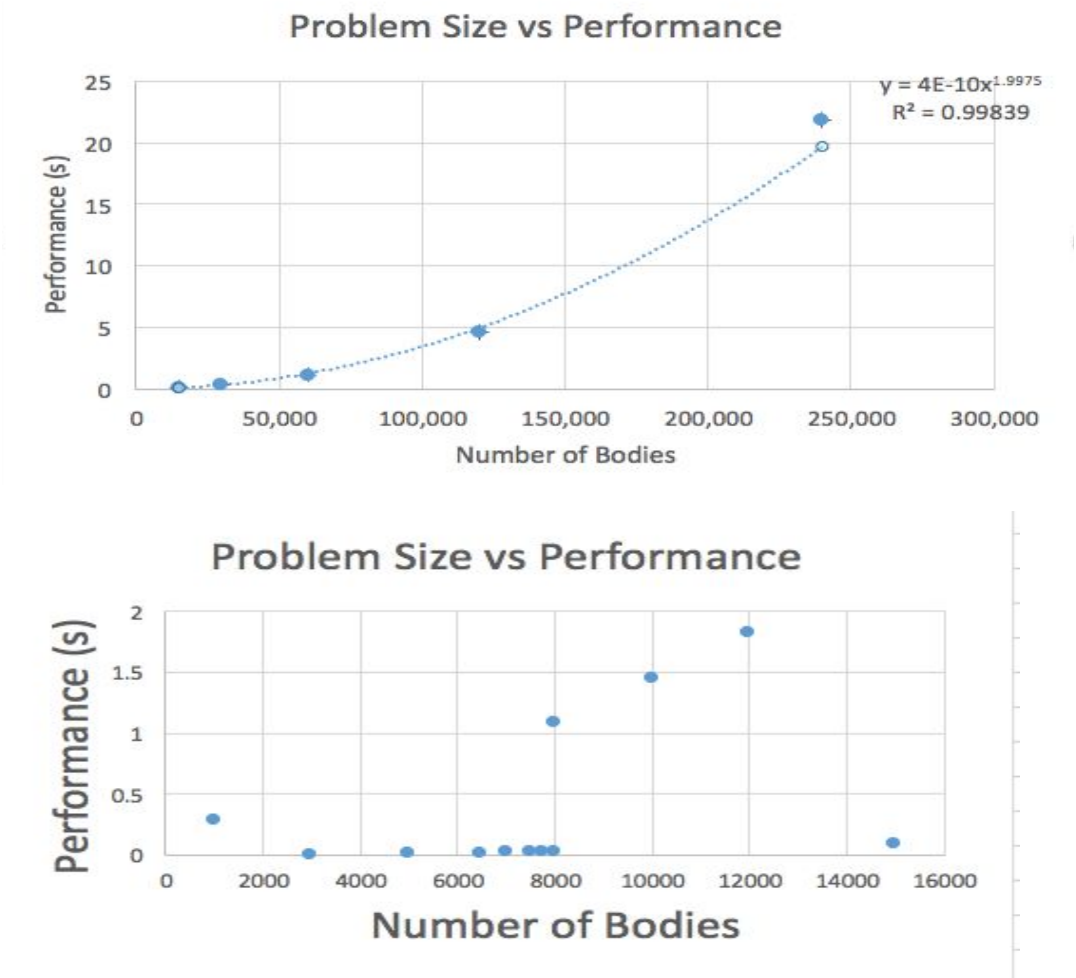


Figure 2: Cuda Program Runtime vs N (zoomed in view of N = 0-16K shown in the bottom).

The performance we observed was indeed O(N^2). However there are interesting spots in the range of 0-15K. One such observation was at N=1K which had a high bump. Since our blocksize was 256, the program had 5 blocks of thread ((2000+256-1)/256). The GPU program is able to handle 8 blocks effectively at once. Hence with 5 blocks of 256 thread each we are essentially underutilizing the GPU. On other points (eg, 5K, 7K, etc) the GPU was able to capitalize on the parallelism effectively making them seem O(1) for the small range. We note that the points 1K, 8K, 10K and 12K follow the general trendline i.e. O(N^2). Beyond 16K we note the dominance of the O(N^2) trend.

4.) Using your answers to the above questions as well as your experience from past labs, try to implement an improvement to the GPU code. Explain how your changes led to an improvement in performance.

Using nvprof we found three sources of inefficiencies: the orginal bodyForce function, the cuda memcopy function (which was being called 20 times) and hidden malloc function.

Modifications:

bodyForce optimization (worked)

We had the best luck optimizing this function (see appendix). We followed Cooper's guidelines where we broke down the single bodyForce into two functions. The new bodyForce acts as a wrapper and instead of looking around all the possible N bodies, does this process in batch with chunkSize of blockIdx.x. Here we are trying to utilize spatial locality in Cuda Memory and Cache architecture. However the speedup is not constant with this approach and the performance is actually worse for N<30K. We had better performance modifying blockSize to 64.
However, we had significant luck with loop unrolling (suggested by Cooper) with about 10% speedup.

The unoptimized code performed as

Avg time: 1.170 ± 0.038s
60000 Bodies: average 27.694 Billion Interactions / second

Optimized Code:

Avg time: 1.054 ± 0.182
60000 Bodies: average 32.099 Billion Interactions / second

Memcopy optimization (failed):

We wanted to take the memcopies out of the for loop (See Appendix 2). Hence we wrote a new bodyForce function that did niterations inside the GPU global function. The results were super fast.

Avg time: 0.00434 ± 0.00097s
60000 Bodies: average 76391 Billion Interactions / second

However, the output p.x, p.y and p.z were not correct. Specifically, each on iteration p.x, p.y and p.z were supposed to be updated with their velocities times dt. However for some reason they did not update and remained at their original values.

Malloc Optimization (failed):

Since we only needed half of the array back (positions and not velocities), we tried allocating a new array that would only be 3*nbodies size. However,for this we had to cuda malloc one additonal array which hindered the gains.

We tried cudaHostAlloc function for pin-to-pin memory transfer. However it didnot work with rand() feature. Although we played around with curand(), during the memcopy, the values were not what we expected (i.e. some of them were nans, etc.)

Finally, we played with openMP and nvcc integration but werent successful.

Appendix:

Mean Values

BlockSize \ nBlocks	1	2	4	8	16	32	64	128	256	512	1024	2048	4096	8192	16384
1	0.0265	0.02716667	0.02716667	0.02616667	0.0275	0.03016667	0.04916667	0.09166667	0.17483333	0.31316667	0.61283333	1.216	2.4285	4.8435	8.856
2	0.0245	0.02516667	0.025	0.02466667	0.02566667	0.02883333	0.046	0.08716667	0.17133333	0.31233333	0.61216667	1.21583333	2.42883333	4.43666667	4.43633333
4	0.02483333	0.02433333	0.024	0.02366667	0.02516667	0.02783333	0.04483333	0.08466667	0.1635	0.31283333	0.6125	1.21683333	2.22383333	2.22333333	2.224
8	0.02433333	0.024	0.02383333	0.0245	0.02483333	0.02933333	0.04516667	0.08283333	0.15933333	0.31116667	0.61316667	1.11983333	1.12	1.1195	1.12066667
16	0.02466667	0.0255	0.02516667	0.0245	0.02533333	0.0285	0.04566667	0.08666667	0.16866667	0.31116667	0.56433333	0.56433333	0.56433333	0.565	0.56483333
32	0.02666667	0.02633333	0.02666667	0.02616667	0.02633333	0.02983333	0.04666667	0.0835	0.1605	0.29216667	0.29166667	0.2915	0.29183333	0.29183333	0.29183333
64	0.02683333	0.02666667	0.027	0.02633333	0.0295	0.04583333	0.08383333	0.16	0.29383333	0.29316667	0.29333333	0.29383333	0.294	0.29383333	0.29383333
128	0.026	0.02583333	0.02616667	0.029	0.04616667	0.084	0.15983333	0.2945	0.295	0.29433333	0.295	0.29383333	0.29433333	0.2945	0.29316667
256	0.027	0.02733333	0.0285	0.04533333	0.08483333	0.16316667	0.2965	0.29633333	0.29783333	0.29666667	0.2965	0.29633333	0.29683333	0.296	0.297
512	0.02733333	0.02766667	0.04633333	0.084	0.15983333	0.2975	0.29616667	0.29783333	0.29766667	0.29833333	0.2995	0.2985	0.2965	0.29733333	0.30016667
1024	0.04633333	0.04566667	0.08416667	0.16016667	0.31233333	0.31233333	0.31283333	0.31266667	0.31266667	0.31266667	0.31283333	0.3125	0.3125	0.3125	0.31216667
2048	0.00766667	0.007	0.00716667	0.00683333	0.00666667	0.00766667	0.0075	0.0075	0.00716667	0.00783333	0.007	0.0075	0.00716667	0.00666667	0.007
4096	0.008	0.00766667	0.00733333	0.00783333	0.00916667	0.0085	0.00766667	0.00816667	0.00733333	0.00783333	0.008	0.00733333	0.00766667	0.00716667	0.00816667
8192	0.00683333	0.007	0.00716667	0.00833333	0.00766667	0.00666667	0.0065	0.0065	0.008	0.00666667	0.009	0.008	0.0075	0.00883333	0.0075
16384	0.009	0.0085	0.00833333	0.00733333	0.0085	0.00766667	0.00866667	0.00833333	0.0085	0.00933333	0.00916667	0.00833333	0.0085	0.00783333	0.00766667

St Dev

BlockSize \ nBlocks	1	2	4	8	16	32	64	128	256	512	1024	2048	4096	8192	16384
1	0.00137841	0.0017224	0.00183485	0.00098319	0.00137841	0.00075277	0.00075277	0.0008165	0.00263944	0.0033116	0.00075277	0.00089443	0.00137841	0.00234521	0.00374166
2	0.00137841	0.00183485	0.00209762	0.00150555	0.00242212	0.00183485	0.00275681	0.00397073	0.00814043	0.00301109	0.00040825	0.00075277	0.00183485	0.00216025	0.00163299
4	0.00132916	0.00136626	0.00063246	0.0008165	0.00116905	0.00098319	0.00075277	0.00413118	0.00703562	0.00426224	0.00054772	0.00075277	0.00098319	0.0008165	0.00154919
8	0.00150555	0.00063246	0.00040825	0.00054772	0.00040825	0.00163299	0.00075277	0.00075277	0.0005164	0.00098319	0.00098319	0.00075277	0.00178885	0.00054772	0.00150555
16	0.00121106	0.00176068	0.00183485	0.00104881	0.0010328	0.00273861	0.00314113	0.00500666	0.00755425	0.00075277	0.0005164	0.0005164	0.0010328	0.00141421	0.00098319
32	0.00150555	0.0008165	0.0010328	0.00075277	0.0010328	0.00183485	0.00163299	0.00083666	0.00083666	0.00098319	0.0005164	0.00083666	0.00040825	0.00075277	0.00040825
64	0.0017224	0.00121106	0.00126491	0.00136626	0.00104881	0.00098319	0.00098319	0.00089443	0.00098319	0.00116905	0.0008165	0.00116905	0.00089443	0.00116905	0.00040825
128	0.00063246	0.00075277	0.00116905	0.00141421	0.0017224	0.00141421	0.00040825	0.00176068	0.00141421	0.00216025	0.00126491	0.00147196	0.00265832	0.00054772	0.00075277
256	0.00154919	0.00136626	0.00137841	0.0005164	0.00453505	0.00778246	0.00207364	0.00163299	0.00132916	0.00163299	0.00242899	0.00121106	0.00147196	0.00189737	0.00228035
512	0.0008165	0.0005164	0.00136626	0.00109545	0.00075277	0.00164317	0.00132916	0.00278687	0.00287518	0.00258199	0.00250998	0.00151658	0.00288097	0.00250333	0.00204124
1024	0.00136626	0.0005164	0.00040825	0.00040825	0.0005164	0.0005164	0.00040825	0.0005164	0.0005164	0.0005164	0.00040825	0.00054772	0.00054772	0.00054772	0.00040825
2048	0.00175119	0	0.00075277	0.00040825	0.0005164	0.0018619	0.00176068	0.00176068	0.00147196	0.00183485	0.00167332	0.00176068	0.00116905	0.0005164	0.00154919
4096	0.00126491	0.00175119	0.00136626	0.00183485	0.00194079	0.00216795	0.0010328	0.00183485	0.0010328	0.0017224	0.00219089	0.0005164	0.00121106	0.00098319	0.00183485
8192	0.00160208	0.00063246	0.00098319	0.00216025	0.00150555	0.0008165	0.00054772	0.00054772	0.00126491	0.0005164	0.00209762	0.00167332	0.00104881	0.00204124	0.00137841
16384	0.00219089	0.00176068	0.00196638	0.0010328	0.00242899	0.00196638	0.00225093	0.00206559	0.00151658	0.00225093	0.00231661	0.00163299	0.00234521	0.0017224	0.00163299
	0.002	0.00083666	0.0010328	0.0005164	0.00176068	0.00040825	0.00176068	0.00136626	0.00176068	0.00225093	0.00126491	0.00136626	0.00231661	0.00098319	0.0018619

Fig: Mean(Top) Performace & Standard Deviation (Bottom) of 6 tests with varying BlockSize and numBlocks

Code Modifications:

1)

```
__device__
void calcForce(Body bi, Body bj, Force* f)
{
     float dx = bj.x - bi.x;
     float dy = bj.y - bi.y;
     float dz = bj.z - bi.z;
     float distSqr = dx*dx + dy*dy + dz*dz + SOFTENING;
     float invDist = rsqrtf(distSqr);
     float invDist3 = invDist * invDist * invDist;
     f->x += dx * invDist3; f->y += dy * invDist3; f->z += dz * invDist3;
}

__global__
void bodyForce(Body *p, float dt, int n) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;
  int chunkSize = blockDim.x;
  Body thisBody = p[i];
  if (i < n) {
     Force f;
     f.x = 0.0f;
     f.y = 0.0f;
     f.z = 0.0f;
     int chunk = 0;
  #pragma unroll
     for (int j = 0; j < n; j+=chunkSize) {
  #pragma unroll
         for (int k = 0; k < chunkSize; k++){
             int idx = j*chunkSize + k ;
             calcForce(p[i], p[idx], &f);
         }
     }

     thisBody.vx += dt*f.x;
     thisBody.vy += dt*f.y;
     thisBody.vz += dt*f.z;
     thisBody.x += thisBody.vx*dt;
     thisBody.y += thisBody.vy*dt;
     thisBody.z += thisBody.vz*dt;

    p[i] = thisBody;
  }
}
```

2)

```
From -->
for (int iter = 1; iter <= nIters; iter++) {
    StartTimer();

    cudaMemcpy(d_buf, buf, bytes, cudaMemcpyHostToDevice);
    bodyForce<<<nBlocks, BLOCK_SIZE>>>(d_p, dt, nBodies); // compute interbody forces
    cudaMemcpy(buf, d_buf, bytes, cudaMemcpyDeviceToHost);
```

To→

```
cudaMemcpy(d_buf, buf, bytes, cudaMemcpyHostToDevice);
for (int iter = 1; iter <= nIters; iter++) {
   StartTimer();

            <code>

}
bodyForce<<<nBlocks, BLOCK_SIZE>>>(d_p, dt, nBodies); // compute interbody forces
cudaMemcpy(buf, d_buf, bytes, cudaMemcpyDeviceToHost);
```

3)

```
__global__
void bodyForce(Body *p, float dt, int n) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;
  const int nIters = 10;  // simulation iterations
  for (int iter = 1; iter <= nIters; iter++) {
    if (i < n) {
    float Fx = 0.0f; float Fy = 0.0f; float Fz = 0.0f;
    Body thisBody = p[i];
    for (int j = 0; j < n; j++) {
       float dx = p[j].x - thisBody.x;
       float dy = p[j].y - thisBody.y;
       float dz = p[j].z - thisBody.z;
       float distSqr = dx*dx + dy*dy + dz*dz + SOFTENING;
       float invDist = rsqrtf(distSqr);
       float invDist3 = invDist * invDist * invDist;

       Fx += dx * invDist3; Fy += dy * invDist3; Fz += dz * invDist3;
     }

     thisBody.vx += dt*Fx;
     thisBody.vy += dt*Fy;
     thisBody.vz += dt*Fz;
     p[i].x += thisBody.vx*dt;
     p[i].y += thisBody.vy*dt;
     p[i].z += thisBody.vz*dt;
     }
   }
}
```

References:

https://www.bu.edu/pasi/files/2011/07/Lecture102.pdf